01 / РЕШЕНИЕ

Java. Видеть. Менять. Проверять.

Воспроизводимый инструмент для анализа и преобразования Java-монолита с проверкой наблюдаемого поведения.

Один исходник. Два представления.

Настоящий AST javac связывает код и визуальную модель. Любое изменение проходит отдельную компиляцию и сравнение вывода обеих версий.

1052

строки в тестовом монолите

22

интеграционных сценария

0

внешних библиотек / API-ключей

02 / АРХИТЕКТУРА

Проверяемый путь от исходника до результата

HTML / CSS / JavaScript + Java 21. Сервер и стандартная библиотека JDK, без облачного AI в критическом контуре.

01

JavaModel

JavacTask.parse()

UTF-16 SourcePositions

Классы, методы, диагностики

→

02

Граф и преобразования

GraphBuilder + Analyzer

Правка исходного span

Оптимизация и extraction

→

03

Runner / quality gate

Две независимые сборки

Новая JVM для каждого stdin

stdout + stderr + exitCode

Истина - Java-текст. Граф хранит точные диапазоны. После правки код повторно разбирается; нетронутые комментарии и форматирование сохраняются.

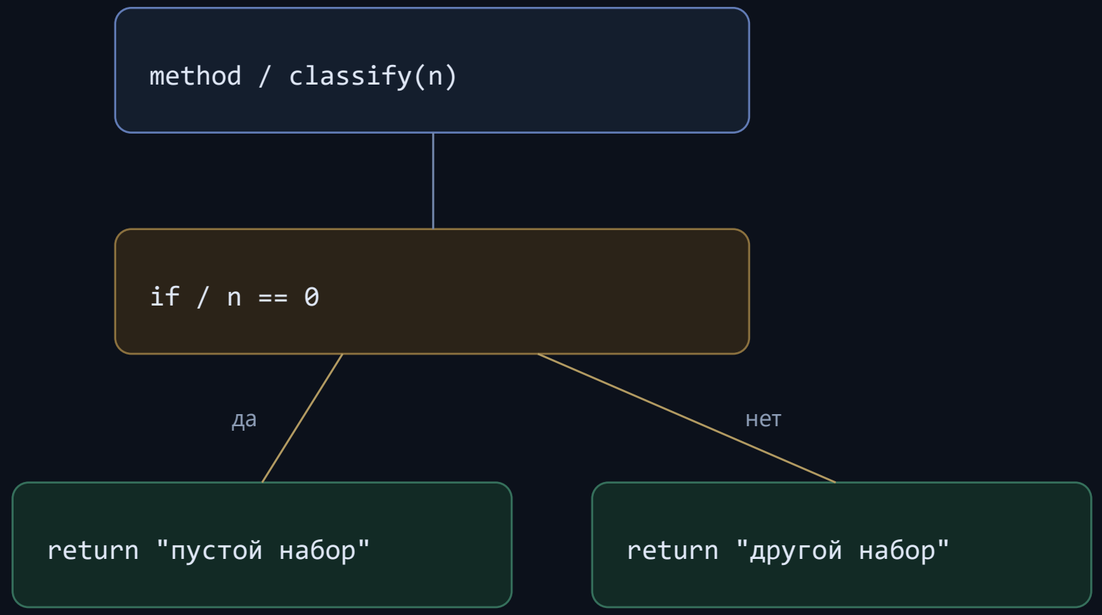
03 / ДВУСТОРОННЯЯ СВЯЗЬ

Изменение узла становится изменением Java

Условия, инструкции и выражения редактируются по исходным диапазонам. Нет промежуточного игрушечного языка.

JAVA / classify()

```
static String classify(int n) {
    if (n == 0) {
        return "пустой набор";
    }
    return "другой набор";
}
```



Произвольная перекоммутация рёбер не поддерживается. Перемещение узла меняет раскладку.

04 / ОПТИМИЗАЦИЯ

Из двух вложенных обходов - индекс частот

Автозамена доступна только для точного чистого AST-шаблона подсчёта упорядоченных равных пар в int[].

ДО / O(n^2)

```
int matches = 0;
for (int a : values) {
    for (int b : values) {
        if (a == b) matches++;
    }
}
return matches;
```

ПОСЛЕ / O(n), СРЕДНЕЕ

```
Map<Integer, Integer> freq = new HashMap<>();
for (int x : values)
    freq.merge(x, 1, Integer::sum);
int matches = 0;
for (int f : freq.values()) matches += f * f;
return matches;
```

```
pairs = sum(f(v) * f(v))
```

Сохранены самопары и модульная int-арифметика. Цена: O(k) памяти. Сторонние эффекты в теле отклоняют шаблон.

05 / QUALITY GATE

Зелёный статус появляется после выполнения

Сравниваются хеши исходных байтов stdout и stderr и код завершения; UTF-8 используется для отображения.

РЕАЛЬНЫЙ ДЕМОНСТРАЦИОННЫЙ ПРОГОН

[OK] Пустой набор / один элемент

[OK] Повторяющиеся и разные значения

[OK] Отрицательные числа / границы int

[OK] Некорректный размер / неполный ввод

8 / 8 входов: поведение совпало

- **Отдельная JVM**

  Каждый тест получает новый процесс и закрытый после передачи stdin.

- **Нет ложного PASS**

  Ошибка сборки, таймаут и лимит вывода отклоняют проверку.

- **Честный FAIL**

  Условие n == 0 → n == 1 меняет результат; gate обнаруживает это.

Конечный набор входов подтверждает поведение на этих данных. Это не формальное доказательство эквивалентности всех Java-программ.

06 / ДЕКОМПОЗИЦИЯ

Выделение ответственности с проверкой зависимостей

Независимый static-метод переносится в BlueprintFunctions.java; исходная сигнатура сохраняется через delegate.

PairAnalyzer.java

```
static int countPairs(int[] values) {
    return BlueprintFunctions
        .countPairs(values);
}
```

→

BlueprintFunctions.java

```
final class BlueprintFunctions {
    static int countPairs(int[] values) {
        // перенесённая реализация
    }
}
```

Сначала компилируется весь проект. Поле или приватный тип, недоступные из helper, отклоняют перенос. Затем нужен quality gate: успешная сборка сама по себе не доказывает сохранение поведения.

07 / ВЕРИФИКАЦИЯ

Проверены успехи и отказные сценарии

Интеграционные тесты обращаются к реальному HTTP API, компилятору и дочерним JVM. Без mock-runner.

Язык и модель

Классы, интерфейсы, циклы, switch, коллекции, checked exceptions, record, generics, lambda, Stream.

Правки и преобразования

UTF-16 с emoji, no-op round trip, редактирование условия, вставка/удаление, extraction и отказ из-за поля.

Сохранение поведения

Случайные массивы с seed 75000, пустые данные, отрицательные числа, границы int, stdout/stderr/exitCode.

Защита от ложного результата

Ошибка компиляции, бесконечный цикл, flood вывода, разные raw bytes, пустой набор входов, traversal и чужой Origin.

22 / 22 сценариев: PASS. Монолит: 1052 строки, 51 метод.

08 / ГРАНИЦЫ

Результат можно проверить и воспроизвести

Сильное решение явно отделяет реализованные гарантии от следующих этапов разработки.

- **Полная Java-семантика**

  Код исполняется стандартной JVM. Граф - структурная проекция, а не альтернативный интерпретатор.

- **Big O - эвристика**

  Линейность циклов предполагается. Вызовы и фактические границы требуют отдельного анализа.

- **Не универсальный рефакторинг**

  Один строгий rewrite-шаблон и перенос независимых static-методов; произвольный граф не генерируется.

- **Exception flow**

  Try/catch/finally, метки и сложные выражения сохраняются; CFG не моделирует все исключительные переходы.

- **Trusted local ≠ sandbox**

  Локальный runner запускает свои исходники. Docker runner подготовлен, но здесь не проверен.

- **Развёрнутый Sites = обзор**

  Сохранённые AST и реальные результаты явно помечены. Для произвольного Java нужна локальная JVM.

09 / ЗАПУСК И ЗАЩИТА

От папки проекта до живого демо - одна команда

Без Maven, npm, ключей API и внешних сервисов во время демонстрации.

WINDOWS

```
.\start.ps1 -TrustedLocal     →     http://localhost:8080
```

01 / Показать проблему — Импорт монолита → countPairs → два вложенных цикла и диагноз.

02 / Улучшить и проверить — Оптимизация → новый граф → 8 реальных входов → PASS.

03 / Показать честный отказ — Правка n == 0 на n == 1 → FAIL → отмена → extraction → экспорт.

Материалы: README.md · docs/ARCHITECTURE.md · docs/SCOPE.md · docs/DEMO.md · исходники · автотесты · JSON-отчёты.